Team-Reflexion und Projektbericht: Automatisierte Browser-Tests mit Playwright, Page Object Model, Maven und KI

Projekt: Web-Anwendung-Test (Playwright Java, JUnit 5, Maven, Page Object Model, BDD/Gherkin)
Team:

- **Hammy:** Konzeption und Umsetzung der HTML/JS-Oberflächen (index.html, confirmation.html) sowie Implementierung der konkreten Page-Object-Klassen in Java (IndexPageObjectModel.java, ConfirmationPageObjectModel.java).
- **Wessam :** Gherkin-Spezifikationen, Implementierung der JUnit-5-Testsuite (WebInterfaceTest.java), Browser-Lifecycle (PlaywrightAbstractTester.java) und Verwaltung des Maven-Builds (pom.xml).
- **Gemeinsame Kernkomponente:** Enge Zusammenarbeit insbesondere bei AbstractPageObjectModel.java, da diese Basisklasse das direkte Fundament und die Pfadauflösung für IndexPageObjectModel und ConfirmationPageObjectModel bildet, sowie bei der finalen Integration und den Erweiterungen nach der Präsentation.

1. Ausgangspunkt, Zusammenarbeit und Umsetzung des Feedbacks aus der heutigen Präsentation

Zu Beginn des Projekts haben wir die Aufgabenbereiche zwischen HTML/Page-Objects (Hammy) und JUnit-5/Maven (Wessam) aufgeteilt, dabei aber von Anfang an eng zusammengearbeitet. Besonders bei der Konzeption von AbstractPageObjectModel.java haben wir gemeinsam gearbeitet: Da sowohl IndexPageObjectModel als auch ConfirmationPageObjectModel direkt von dieser Klasse erben (extends AbstractPageObjectModel) und über getProjectRoot() ihre lokalen file:///-URLs beziehen, musste diese Schnittstelle für beide Seiten exakt abgestimmt sein.
In unserer ersten Version vor der Präsentation stand das Grundgerüst mit den beiden Webseiten index.html und confirmation.html unter src/main/webapp/, den Page-Object-Klassen sowie den ersten Basistests (TC1 bis TC4). Im Rahmen unserer heutigen Präsentation erhielten wir von Herrn Prof. Dr. Brocks gezieltes Feedback für die finale Abgabe (ergänzend zu den Hinweisen aus der vorherigen *Thermometer*-Aufgabe bezüglich assertAll und einer ausführlicheren KI-Reflexion): Wir sollten zusätzlich den Standardwert „Paris“ der Combobox prüfen, ein vollständig ausgefülltes Formular visuell und farblich (Grün vs. Rot) von einem unvollständigen Formular unterscheiden, die Auswirkung der Checkbox durch eine eigene Textmeldung („Abonnent“ vs. „Nicht-Abonnent“) überprüfbar machen, zusammengehörige Prüfungen mit assertAll bündeln und einen parametrisierten Test (@ParameterizedTest) ergänzen.

Diese Punkte haben wir unmittelbar nach der Präsentation gemeinsam umgesetzt:

1. **Erweiterung der HTML/JS-Logik und der Java-Page-Objects (Hammy):**
   In index.html wurde die Inline-JavaScript-Funktion afficherResultat() erweitert. Ein vollständig ausgefülltes Formular gibt nun eine grüne Erfolgsmeldung (.succes, rgb(0, 128, 0)) aus, während ein fehlender oder nur aus Leerzeichen bestehender Name (nom.trim() === "") eine rote Fehlermeldung (.erreur, rgb(255, 0, 0)) erzeugt. Zudem wurde das Element <p id="message_alertes"> hinzugefügt, das abhängig vom Status der Checkbox #checkbox_newsletter dynamisch „*Vous recevrez nos alertes.*“ oder „*Vous ne recevrez pas d'alertes.*“ anzeigt (bzw. bei einem Fehler leer bleibt). Passend dazu wurde IndexPageObjectModel.java um die Methoden recupererMessageAlertes(), getMessageColor() (mittels evaluate("el => getComputedStyle(el).color")), getVilleSelectionnee() (mittels inputValue()) und getHeadingText() ergänzt. Dadurch bleibt das Page Object Pattern strikt gewahrt, ohne dass in den Tests direkt auf den DOM oder auf CSS-Selektoren zugegriffen wird.

2. **Konzeptionelle Bündelung mit assertAll (Wessam & Hammy):**
   Entsprechend dem Feedback aus der *Thermometer*-Korrektur und der Präsentation wurden mehrteilige Überprüfungen eines Seitenzustands in WebInterfaceTest.java konsequent in benannten assertAll-Blöcken zusammengefasst (TC1, TC2c, TC3, TC4 und TC6). Allgemeine Formulierungen wie „die Bestätigungsseite öffnet sich“ wurden in präzise Einzel-Assertions übersetzt (getUrl().endsWith(...), getTitle() und getHeadingText()). Schlägt eine Teilprüfung fehl, bricht der Test nicht sofort ab, sondern berichtet alle fehlerhaften Eigenschaften auf einmal.

3. **Neue Testfälle (TC2b, TC2c, TC5) und @ParameterizedTest (TC6) (Wessam):**
   Neben dem Test für das explizite An- und Abhaken der Checkbox (TC2b) und der Fehlerprüfung bei leerem Pflichtfeld (TC2c) überprüft TC5 nun gezielt, dass beim Laden der Seite standardmäßig „Paris“ in der Combobox ausgewählt ist. Um das DRY-Prinzip (*Don't Repeat Yourself*) einzuhalten, wurde über junit-jupiter-params in TC6 ein @ParameterizedTest mit @CsvSource implementiert. Dieser prüft sieben Kombinationen aus Benutzernamen, Städten (Paris, Berlin, Metz), Checkbox-Zuständen (true/false), erwarteten CSS-Farben (GREEN/RED) sowie Randfällen (leerer String '' und Leerzeichen ' '). Insgesamt laufen damit alle 14 Tests über mvn test fehlerfrei durch (BUILD SUCCESS).

2. Gemeinsame technische Hürden: Maven, Plattformunterschiede und Git-Integration

Die größte praktische Schwierigkeit für uns beide lag weniger in der Programmierlogik als im Verhalten von **Maven** und den Unterschieden zwischen unseren Systemumgebungen:

- **Instabilitäten mit Maven und IDE-Synchronisation:**
  Sowohl in IntelliJ IDEA als auch in VS Code erlebten wir Situationen, in denen Maven scheinbar unvorhersehbar funktionierte und kurz darauf Fehler meldete. Nach dem Klonen des Repositories wurden JUnit-5- und Playwright-Importe (org.junit.jupiter.api, com.microsoft.playwright.Page) zunächst nicht erkannt, weil die IDE das Projekt nicht automatisch als Maven-Projekt indiziert hatte und src/test/java manuell als *Test Sources Root* zugewiesen werden musste. Auch sporadische Cache-Probleme des Compilers (I/O error reading dependency state: ...\target\classes) ließen sich erst durch ein konsequentes mvn clean test und ein *Rebuild Project* beheben.
- **Plattformübergreifende Pfadauflösung in AbstractPageObjectModel:**
  Bei unserer gemeinsamen Arbeit an AbstractPageObjectModel.getProjectRoot() zeigte sich, dass Windows-Pfade mit Backslashes (\) und führenden Laufwerksbuchstaben (/C:/...) beim Aufruf von open() in IndexPageObjectModel und ConfirmationPageObjectModel zu ungültigen file:///-URLs führten. Durch das Ersetzen der Backslashes (replace("\\", "/")) und das Entfernen des führenden Schrägstrichs vor Windows-Laufwerksbuchstaben (^/[a-zA-Z]:.*) konnten wir sicherstellen, dass die Seiten sowohl unter Windows als auch unter Linux (GitHub Codespaces) zuverlässig geladen werden.
- **Kommandozeilen-Unterschiede und Headless-Modus (Windows vs. Codespaces):**
  Unter Windows war mvn anfangs nicht global im PATH eingetragen, und der Befehl mvn test -Dplaywright.headless=false führte in der Windows PowerShell zu einer LifecyclePhaseNotFoundException, weil PowerShell das Argument am Punkt auftrennt, sofern es nicht in Anführungszeichen steht ("-Dplaywright.headless=false"). Zudem mussten wir darauf achten, dass der Standardwert in PlaywrightAbstractTester.java für GitHub Codespaces und die Abgabe auf headless = true stehen muss, da ein Linux-Container ohne grafischen X-Server andernfalls mit einer MojoFailureException abstürzt.
- **Git-Branching und Merge-Auflösung:**
  Unsere parallele Arbeit auf main und dem Branch tests-binome2 führten wir nach den letzten Anpassungen über git merge origin/tests-binome2 zusammen. Den dabei entstandenen Merge-Konflikt in WebInterfaceTest.java haben wir gemeinsam aufgelöst und anschließend ungenutzte private Konstanten (ALERTS_ON, ALERTS_OFF) sowie redundante Imports entfernt, um einen sauberen Code ohne Warnungen abzugeben.

3. Kritische Reflexion zu BDD/Gherkin und dem Einsatz von KI-Werkzeugen

Mehrwert von Gherkin und POM im Team

Die Dokumentation jedes Testfalls mit Gherkin (Feature / Scenario / Given / When / Then) diente uns als klare Schnittstelle zwischen Hammy (HTML & Page Objects) und Wessam (JUnit-Tests). Da jede Gherkin-Zeile in WebInterfaceTest.java direkt einem Methodenaufruf in IndexPageObjectModel oder ConfirmationPageObjectModel entspricht, ließen sich die heutigen Erweiterungen nach der Präsentation schnell und fehlerfrei integrieren.

Kritische Würdigung des KI-Einsatzes

- **Wo die KI uns konkret geholfen hat (Stärken):**
  - *Umsetzung spezifischer Prüfungen:* Um die heute geforderte Prüfung der grünen und roten Textfarbe in Playwright sauber umzusetzen, lieferte die KI den Hinweis auf evaluate("el => getComputedStyle(el).color") in IndexPageObjectModel. Ebenso half sie bei der übersichtlichen Gestaltung der @CsvSource-Tabelle mit Pipe-Delimitern (|) in TC6.
  - *Fehlerdiagnose bei Build- und Terminal-Problemen:* Bei den kryptischen Fehlermeldungen von Maven, Surefire und der Windows PowerShell half die KI dabei, schnell zwischen echten Codefehlern und reinen Umgebungs- bzw. Quoting-Problemen zu unterscheiden.
- **Wo die KI an Grenzen stieß und eigene Korrekturen nötig waren (Schwächen):**
  - *Fehlendes Bewusstsein für die Laufzeitumgebung:* Auf die Frage, warum sich in GitHub Codespaces kein Browserfenster öffnet, schlug die KI pauschal vor, playwright.headless auf false zu setzen. Da Codespaces jedoch in einem Headless-Container ohne Grafikserver läuft, bringt dieser Vorschlag den gesamten Testlauf zum Absturz. Ohne unser eigenes Verständnis der Umgebung hätte uns dieser Rat blockiert.
  - *Toter Code nach dem Refactoring auf @ParameterizedTest:* Beim Umbau auf den parametrisierten Test TC6 blieben durch KI-gestützte Zwischenschritte ungenutzte Konstanten (ALERTS_ON, ALERTS_OFF) und vollqualifizierte Aufrufe (org.junit.jupiter.api.Assertions.assertFalse) im Code zurück, weil Java innerhalb eines @CsvSource-Textblocks keine Variablenreferenzen auflösen kann. Erst unsere eigene Kontrolle der IDE-Warnungen deckte diesen toten Code auf.
  - *Unnötige Komplexität:* KI-Vorschläge enthielten teilweise explizite Wartebefehle (Thread.sleep oder manuelle Timeouts), obwohl Playwrights integriertes *Auto-Waiting* auf locator()-Ebene die Synchronisation bereits vollständig übernimmt. Solche Zusätze haben wir bewusst weggelassen, um den Code schlank und wartbar zu halten.

4. Fazit

Durch unsere enge Abstimmung bei AbstractPageObjectModel und die saubere Trennung zwischen HTML-Ansicht, Page Objects und JUnit-5-Tests konnten wir alle Rückmeldungen aus der heutigen Präsentation zügig umsetzen, ohne bestehende Funktionen zu beschädigen. Trotz der anfänglichen Hürden mit Maven und den Betriebssystem-Unterschieden verfügen wir nun über eine vollständig dokumentierte, parametrisierte und reproduzierbare Testsuite.